

# Suport curs algoritmica grafurilor

## IV. Programare liniara, Drumuri minime între toate perechile de vârfuri

### 4.1 Programare Liniara

**Problema generală:** fie o matrice  $A$  de dimensiune  $m \times n$ , un vector  $b$  de dimensiune  $m$  și un vector  $c$  de dimensiune  $n$ . Trebuie găsit un vector  $x$  de  $n$  elemente care maximizează funcția obiectiv

$$\sum_{i=1}^n c_i x_i$$

și satisface  $m$  constrângeri date de

$$Ax \leq b.$$

În unele cazuri nu prezintă interes funcția obiectiv, se dorește găsirea unei soluții fezabile (orice vector  $x$  ce satisface  $Ax \leq b$ ) sau sa se arate că nu există astfel de soluții.

Într-un sistem de constrângeri fiecare rând din matricea  $A$  conține o valoare  $-1$ , o valoare  $1$  și restul valorilor sunt  $0$ . Astfel constrângerile date de  $Ax \leq b$  sunt un set de  $m$  constrângeri cu  $n$  necunoscute unde fiecare constrângere este o inecuație de forma

$$x_j - x_i \leq b_k,$$

unde  $1 \leq i, j \leq n, i \neq j$  și  $1 \leq k \leq m$ .

Exemplu

$$\begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} \leq \begin{pmatrix} 0 \\ -1 \\ 1 \\ 5 \\ 4 \\ -1 \\ -3 \\ -3 \end{pmatrix}$$

problema cere să se găsească  $x_1, x_2, x_3, x_4, x_5$  pentru cele 8 constrângeri

$$x_1 - x_2 \leq 0,$$

$$x_1 - x_5 \leq -1,$$

...

Soluția nu este unică, două posibile soluții:

$$x = (-5, -3, 0, -1, -4)$$

$$x' = (0, 2, 5, 4, 1)$$

.

#### 4.1.1 Constrângeri sub forma unui graf

Cum se poate modela problema sub forma unui graf?

**Grafuri de constrângeri** sistemul de constrângeri poate fi interpretat sub forma unui graf, pentru un sistem  $Ax \leq b$  de constrângeri, matricea  $A$  de dimensiune  $m \times n$  poate fi văzută ca transpusa unei matrici de incidență a unui graf cu  $n$  vârfuri și  $m$  arce. Fiecare vârf  $v_i \in V, i = 1, 2, \dots, n$  corespunde unei variabile  $x_i$ . Fiecare arc  $(i, j) \in E$  corespunde unei inegalități

Fie un sistem  $Ax \leq b$  de constrângeri, graful corespunzător acestui sistem este un graf ponderat și orientat  $G = (V, E)$  unde  $V = \{v_0, v_1, \dots, v_n\}$  și

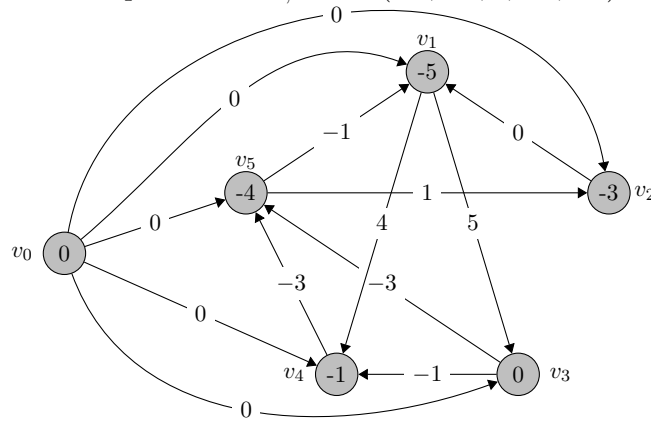
$$E = \{(v_i, v_j) \mid x_j - x_i \leq b_k \text{ este o constrângere}\} \cup \{(v_0, v_1), (v_0, v_2), \dots, (v_0, v_n)\}$$

Fie un sistem  $Ax \leq b$  de constrângeri și  $G = (V, E)$  graful constrângerilor. Dacă  $G$  nu conține circuite de pondere negativă, atunci

$$x = (\delta(v_0, v_1), \delta(v_0, v_2), \dots, \delta(v_0, v_n))$$

este o soluție fezabilă pentru sistem. Dacă graful  $G$  conține un circuit negativ, sistemul nu are soluție.

Exemplu: pentru sistemul definit mai sus se poate desena graful de mai jos, în graf valoarea  $\delta(v_0, v_i)$  apare în fiecare nod. O posibilă soluție  $x = (-5, -3, 0, -1, -4)$ .



#### Drumuri minime

Problema găsirii drumului minim între toate perechile de vârfuri ale unui graf:

- **se dă:** un graf orientat  $G = (V, E)$  cu funcția de pondere  $\omega : E \rightarrow \mathbb{R}$ ,
- **se vrea:** să se găsească pentru fiecare pereche de vârfuri  $u, v \in V$  un drum minim de la  $u$  la  $v$  (suma ponderilor arcelor din drum să fie minimă),

- afișarea rezultatului se face sub forma unei matrici  $A_{n,n}$ ,  $n = |V|$ , unde elementul  $a_{i,j} = \delta(i,j)$  arată lungimea drumului de la vârful  $i$  la vârful  $j$ .

Idee: se poate rezolva problema dacă se rulează un algoritm de drum minim între un vârf sursă  $s$  și toate vârfurile din graf  $V \setminus \{s\}$  de  $|V|$  ori (pentru fiecare vârf din graf ca și sursă). De exemplu:

- dacă există ponderi negative se rulează BELLMAN\_FORD pentru fiecare vârf din graf, complexitatea  $O(V^2E)$  iar dacă graful este dens  $O(V^4)$ ;
- dacă toate ponderile nu sunt negative se poate rula Dijkstra pentru fiecare vârf din graf, complexitatea  $O(V \lg V)$  dacă se folosește un binary heap pentru coada de priorități ( $O(V^3 \lg V)$  pentru un graf dens) și  $O(V^2 \lg V + VE)$  dacă se folosește un Fibonacci heap pentru coada de priorități ( $O(V^3)$  dacă graful este dens).

Vom vedea cum putem determina drumul de cost minim între oricare două vârfuri din graf în  $O(V^3)$  în toate cazurile, fără a folosi structuri de date speciale.

## 4.2 Drumuri minime și înmulțirea unor matrici

Majoritatea algoritmilor își reprezintă un graf folosind ca și reprezentare matricea de adiacență. Fie un graf  $G = (V, E)$ , vârfurile sunt numerotate de la 1 la  $n$ , reprezentat de o matrice de adiacență de ponderi  $A = (a_{i,j})_{i,j=1,\overline{n}}$  unde:

$$a_{i,j} = \begin{cases} 0 & \text{dacă } i = j, \\ \text{ponderea arcului } (i,j) & \text{dacă } i \neq j, (i,j) \in E, \\ \infty & \text{dacă } i \neq j, (i,j) \notin E. \end{cases}$$

Rezultatul va fi matricea  $D = (d_{i,j})$ , unde  $d_{i,j} = \delta(i,j)$ .

Se prezintă o soluție bazată pe programare dinamică pentru a rezolva problema drumului minim între oricare două vârfuri din graf.

Pentru aceasta trebuie:

1. caracterizată structura unei soluții optimale
2. definită recursiv valoarea soluției optimale
3. calculată valoarea soluției optimale

### 4.2.1 Structura unui drum minim

Pe baza lemei drumului minim (vezi cursurile anterioare): oricare subdrum dintr-un drum minim este drum minim. Graful  $G = (V, E)$  este reprezentat de matricea de adiacență  $A = (a_{i,j})$ .

- fie  $p$  drumul minim de la vârful  $i$  la vârful  $j$  format din  $m$  arcuri;
- dacă nu există un circuit negativ  $m$  este finit;
- dacă  $i = j$  ponderea lui  $p$  este 0;
- dacă  $i \neq j$  atunci  $i \xrightarrow{p'} k \rightarrow j$ , unde drumul  $p'$  conține  $m - 1$  arce;
- din lema drumului minim:  $p'$  este drum minim de la  $i$  la  $k$  și  $\delta(i,j) = \delta(i,k) + a_{k,j}$ .

### 4.2.2 O soluție recursivă

Fie  $l_{ij}^{(m)}$  ponderea minimă a unui drum  $i \rightsquigarrow j$  de  $m$  arce

$$l_{ij}^{(0)} = \begin{cases} 0, & \text{dacă } i = j, \\ \infty, & \text{dacă } i \neq j. \end{cases}$$

De la  $m \geq 1$  se calculează  $l_{ij}^{(m)}$  ca minimul lui  $l_{ij}^{(m-1)}$  (ponderea drumului minim de la  $i$  la  $j$  format din cel mult  $m-1$  arce) și ponderea minimă a oricărui drum de la  $i$  la  $j$  format din  $m$  arce, obținută dacă ne uităm la toți predecesorii  $k$  ai lui  $j$ .

Se pot defini recursiv:

$$l_{ij}^{(m)} = \min(l_{ij}^{(m-1)}, \min_{1 \leq k \leq n} \{l_{ij}^{(m-1)} + a_{ik}\}) = \min_{1 \leq k \leq n} \{l_{ij}^{(m-1)} + a_{jk}\} \quad (1)$$

deoarece  $a_{jj} = 0 \forall j$ .

Care este drumul minim  $\delta(i, j)$ ?

Dacă  $G$  nu are circuite negative, pentru orice pereche  $i$  și  $j$  pentru care  $\delta(i, j) < \infty$  există un drum minim de la  $i$  la  $j$  simplu ce conține cel mult  $n-1$  arce. Deci:

$$\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)}.$$

### 4.2.3 Calculul drumului minim

Ca și date de intrare se primește  $A = (a_{ij})$  și se calculează o serie de matrici  $L^{(1)}, L^{(2)}, \dots, L^{(n-1)}$  unde pentru  $m = 1, 2, \dots, n-1$  avem  $L^{(m)} = (l_{ij}^{(m)})$ .  $L^{(n-1)}$  conține drumul minim (ponderile drumului minim)

$$l_{ij}^{(1)} = a_{ij} \forall i, j \in \mathbb{N} \Rightarrow L^{(1)} = A.$$

Următoarea procedură primește ca și parametrii  $L^{(n-1)}$ ,  $A$  și întoarce  $L^{(n)}$ .

#### EXTINDE\_DRUM\_MINIM(L,A)

```

1:  $n = L.\text{rânduri}$ 
2: fie  $L' = (l'_{ij})$  o matrice  $n \times n$ 
3: for  $1 \leq i \leq n$  do
4:   for  $1 \leq j \leq n$  do
5:      $l'_{ij} = \infty$ 
6:   for  $1 \leq k \leq n$  do
7:      $l'_{ij} = \min(l'_{ij}, l_{ik} + a_{kj})$ 
8: return  $L'$ 
```

Procedura determină matricea  $L'$  folosind relația (1), timpul de execuție este  $\Theta(V^3)$ . Se poate face o paralelă cu procedura de înmulțirea a două matrici.

Practic drumul minim se determină prin extinderea drumului minim arc cu arc. Trebuie să se determine:

$$\begin{aligned} L^{(1)} &= L^{(0)} \cdot A = A \\ L^{(2)} &= L^{(1)} \cdot A = A^2 \\ L^{(3)} &= L^{(2)} \cdot A = A^3 \\ &\dots \\ L^{(n-1)} &= L^{(n-2)} \cdot A = A^{n-1} \end{aligned}$$

Matricea  $L^{(n-1)} = A^{n-1}$  conține drumul minim. Următoarea procedură determină secvența în  $\Theta(V^4)$ .

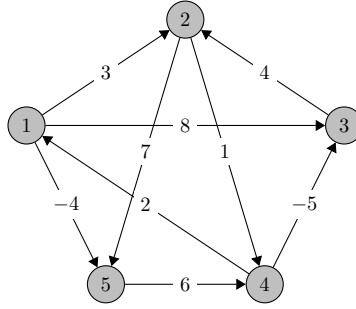


Figura 1: Exemplu determinare drum minim

#### DETERMINA\_TOATE\_DRUMURILE\_MIN(A)

- 1:  $n = A.\text{rânduri}$
- 2:  $L^{(1)} = A$
- 3: **for**  $2 \leq m \leq n - 1$  **do**
- 4:     fie  $L^{(m)}$  o matrice  $n \times n$
- 5:      $L^m = \text{EXTINDE\_DRUM\_MINIM}(L^{m-1}, A)$
- 6: **return**  $L^{(n-1)}$

De exemplu, fie graful din figura 1 pentru care se determină matricea  $L^{(n-1)}$ :

$$L^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

...

$$L^{(4)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

#### 4.2.4 Îmbunătățirea timpului de rulare

Nu trebuie determinate  $m$  matrici, ne interesează doar  $(n - 1)$  matrici.  $L^{(n-1)}$  se poate calcula din  $\lceil \lg(n - 1) \rceil$  pași deoarece  $2^{\lceil \lg(n-1) \rceil} \geq n - 1$  produsul final  $L^{(2^{\lceil \lg(n-1) \rceil})}$  este  $L^{(n-1)}$ .

$$\begin{aligned} L^{(1)} &= A \\ L^{(2)} &= A^2 = A \cdot A \\ L^{(4)} &= A^4 = A^2 \cdot A^2 \\ L^{(8)} &= A^8 = A^4 \cdot A^4 \\ &\dots \\ L^{(2^{\lceil \lg(n-1) \rceil})} &= A^{2^{\lceil \lg(n-1) \rceil}} = A^{2^{\lceil \lg(n-1) \rceil - 1}} \cdot A^{2^{\lceil \lg(n-1) \rceil - 1}} \end{aligned}$$

Procedura este:

#### MAI\_RAPID\_TOATE\_DRUMURILE\_MIN(A)

- 1:  $n = A.\text{rânduri}$

```

2:  $L^{(1)} = A$ 
3:  $m = 1$ 
4: while  $m \leq n - 1$  do
5:   fie  $L^{(2m)}$  o matrice  $n \times x$ 
6:    $L^{2m} = \text{EXTINDE\_DRUM\_MINIM}(L^m, L^m)$ 
7:    $m = 2m$ 
8: return  $L^{(m)}$ 

```

Timpul de execuție este  $\Theta(V^3 \lg V)$ , fiecare produs de matrice ia  $\Theta(V^3)$  datorită  $\lceil \lg(n-1) \rceil$ .

### 4.3 Floyd-Warshall

Algoritmul a fost discutat în cursurile anterioare.

Recursiv, fie  $d_{ij}^{(k)}$  ponderea drumului minim de la  $i$  la  $j$ , vârfurile intermediare drumului sunt în setul  $\{1, 2, \dots, k\}$ .

$$d_{i,j}^{(k)} = \begin{cases} a_{ij} & , \text{dacă } k = 0, \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & , \text{dacă } k \geq 1. \end{cases}$$

### 4.4 Închiderea tranzitivă a unui graf orientat

Fie  $G = (V, E)$  un graf orientat cu setul  $V = \{1, 2, \dots, n\}$ , trebuie să se determine dacă există un drum în graful  $G$  de la vârful  $i$  la vârful  $j$ , unde  $i, j \in V$ .

Închiderea tranzitivă a lui  $G$  este definită ca și graful  $G^* = (V, E^*)$ , unde

$$E^* = \{(i, j) \mid \text{dacă există } i \rightsquigarrow j \text{ în } G\}.$$

**O modalitate** de a determina închiderea tranzitivă în  $\Theta(V^3)$  e de a rula algoritmul *Floyd-Warshall* pe  $G$  cu ponderea 1 pe fiecare arc. Dacă există un drum  $i \rightsquigarrow j$  atunci  $d_{ij} < n$  altfel  $d_{ij} = \infty$ .

Există o **altă metodă** pentru a determina închiderea tranzitivă în  $\Theta(V^3)$  care poate salva timp și spațiu în practică: se substituie operațiile aritmetice  $\min$  și  $+$  din *Floyd-Warshall* cu operațiile logice  $\vee$  (*SAU* logic) și  $\wedge$  (*SI* logic).

Pentru  $i, j, k = 1, \dots, n$  se definește  $t_{ij}^{(k)} = 1$  dacă exista un drum  $i \rightsquigarrow j$  cu vârfurile intermediare în setul  $\{1, \dots, k\}$  și  $t_{ij}^{(k)} = 0$  în rest.

Se construiește închiderea tranzitivă  $G^* = (V, E^*)$  prin adăugarea arcului  $(i, j)$  în  $E^*$  dacă și numai dacă  $t_{ij}^{(k)} = 1$ .

Pentru a calcula recursiv închiderea tranzitivă, putem defini:

$$t_{i,j}^{(0)} = \begin{cases} 0 & , \text{dacă } i \neq j \text{ și } (i, j) \notin E, \\ 1 & , \text{dacă } i \neq j \text{ și } (i, j) \in E, \end{cases}$$

și pentru  $k \geq 1$

$$t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)}).$$

Se determină matricile  $T^k = (t_{ij}^{(k)})$  pentru un  $k \nearrow \nearrow$ . Procedura pentru determinarea închiderii tranzitive este:

#### INCHIDERE\_TRANZITIVA(G)

```

1:  $n = |V|$ 
2: fie  $T^{(0)} = (t_{ij}^{(0)})$  o matrice  $n \times x$ 
3: for  $1 \leq i \leq n$  do
4:   for  $1 \leq j \leq n$  do

```

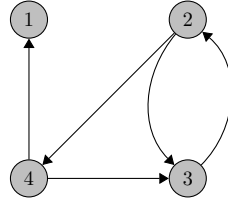


Figura 2: Exemplu determinare închidere tranzitivă

```

5:   if  $i == j$  sau  $(i, j) \in E$  then
6:        $t_{ij}^{(0)} = 1$ 
7:   else
8:        $t_{ij}^{(0)} = 0$ 
9:   for  $1 \leq k \leq n$  do
10:      fie  $T^{(k)} = (t_{ij}^{(k)})$  o matrice  $n \times n$ 
11:      for  $1 \leq i \leq n$  do
12:          for  $1 \leq j \leq n$  do
13:               $t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)})$ 
14:   return  $T^{(m)}$ 
    
```

De exemplu, fie graful din figura 2 pentru care se determină matricile  $T^{(k)}$ :

$$T^{(0)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & \color{red}{1} \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

$$T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & \color{red}{1} & 1 & 1 \end{pmatrix} \quad T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \color{red}{1} & 1 & 1 & 1 \\ \color{red}{1} & 1 & 1 & 1 \\ 1 & \color{red}{1} & 1 & 1 \end{pmatrix}$$

#### 4.5 Algoritmul lui Johnson pentru grafuri rare

Algoritmul găsește drumul minim în  $O(V^2 \lg V + VE)$ . Pentru grafuri rare e asimptotic mai rapid decât înmulțirea de matrici și Floyd-Warshall. Algoritmul găsește o soluție pentru grafuri fără circuite negative. Folosește ca și subrutina *Dijkstra* și *Bellman-Ford*.

Algoritmul folosește tehnica de reponderare:

- dacă toate ponderile sunt strict pozitive pot rula *Dijkstra* din fiecare vârf și pot găsi drumul minim în  $O(V^2 \lg V + VE)$  (Fibonacci heap)
- dacă  $G$  are ponderi negative dar nu are circuit negativ trebuie recalculate ponderile astfel încât să fie pozitive, noul set de ponderi  $\hat{w}$  trebuie să satisfacă următoarele:
  1. pentru toate perechile  $u, v \in V$ , un drum  $p$  este minim de la  $u$  la  $v$  folosind ponderile  $w$  dacă  $p$  e drum minim de la  $u$  la  $v$  și pentru ponderile  $\hat{w}$ ;
  2. pentru toate arcele  $(u, v)$ ,  $\hat{w}(u, v) \geq 0$ ;
- $\hat{w}$  se poate determina în  $O(VE)$ .

Prin reponderare drumul minim trebuie păstrat. Notăm cu:

- $\delta$  drumul minim din ponderile  $w$ ,
- $\hat{\delta}$  drumul minim din ponderile  $\hat{w}$ .

**Lema 4.1 (Reponderarea nu schimbă drumurile minime)** *fie un graf orientat și ponderat  $G = (V, E)$  cu funcția de pondere  $w : E \rightarrow \mathbb{R}$ , fie  $h : V \rightarrow \mathbb{R}$  o funcție ce mapează vârfurile la numere reale. Pentru fiecare arc  $(u, v) \in E$  se definește*

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v).$$

Fie  $p = \langle v_0, \dots, v_k \rangle$  un drum de la  $v_0$  la  $v_k$ ,  $p$  este un drum minim de la  $v_0$  la  $v_k$  cu  $w$  dacă e drum minim și pentru  $\hat{w}$ ,  $w(p) = \delta(v_0, v_k) \Leftrightarrow \hat{w}(p) = \hat{\delta}(v_0, v_k)$ .

**Reponderare pozitivă** se vrea ca  $\hat{w}(u, v) \geq 0$  pentru  $(u, v) \in E$ . Fie  $G = (V, E)$  cu  $w : E \rightarrow \mathbb{R}$ , se construiește un nou graf  $G' = (V', E')$  unde:

- $V' = V \cup \{s\}$ ,  $s \in V$
- $E' = E \cup \{(s, v) | v \in V\}$ ,  $w(s, v) = 0, \forall v \in V$ .

$G'$  nu are circuite negative dacă  $G$  nu are circuite negative. Se definește  $h(v) = \delta(s, v), \forall v \in V'$

$$h(v) \leq h(u) + w(u, v), \forall (u, v) \in E'$$

deci

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0.$$

Algoritmul lui Johnson este:

#### JOHNSON(G)

```

1: determină  $G', V' = V \cup \{s\}, E' = E \cup \{(s, v) | v \in V\}$  și  $w(s, v) = 0 \forall v \in V$ 
2: if  $BELLMAN\_FORD(G', w, s) == FALSE$  then
3:   exit
4: else
5:   for  $v \in V'$  do
6:     pune  $h(v) = \delta(s, v)$  determinată de  $BELLMAN\_FORD$ 
7:   for  $(u, v) \in E'$  do
8:      $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ 
9:   fie  $D = (d_{uv})$  o matrice  $n \times n$ 
10:  for  $u \in V$  do
11:    rulează  $DIJKSTRA(G, \hat{w}, u)$  pentru a determina  $\hat{\delta}(u, v) \forall v \in V$ 
12:    for  $v \in V$  do
13:       $d_{uv} = \hat{\delta}(u, v) - h(u) + h(v)$ 
14:  return  $D$ 

```

Ca și exemplu fie graful  $G'$  cu funcția de pondere  $w$  din figura 4. Figura 5 prezintă graful după reponderare. Figurile 6a-6e prezintă rezultatul rulării algoritmului *Dijkstra* pentru fiecare vârf din  $G$  ca și sursă, vârfurile sursă este negru iar drumul minim este reprezentat de arcele marcate cu gri. Fiecare vârf conține valorile  $\hat{\delta}(u, v) / \delta(u, v)$ . Valoarea  $d_{uv} = \delta(u, v)$  este  $\hat{\delta}(u, v) + h(u) - h(v)$ .

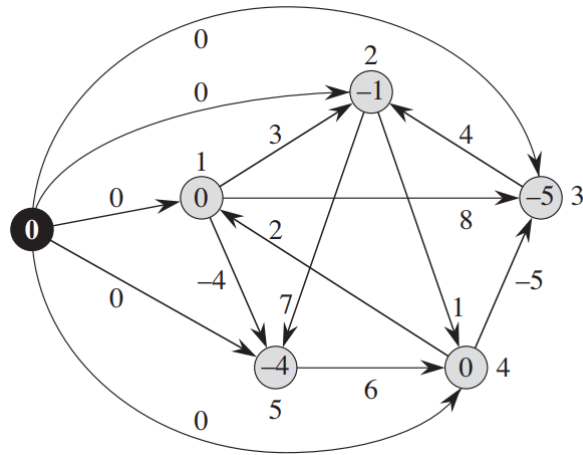


Figura 4: Graful  $G'$  cu funcția de pondere  $w$ .

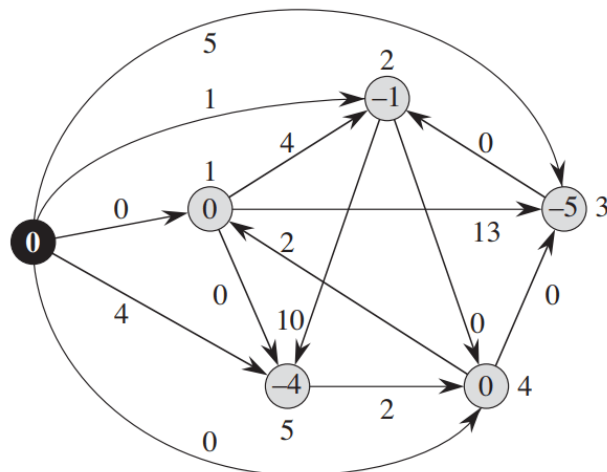


Figura 5: Graful  $G'$  cu funcția de pondere  $\hat{w}$ .

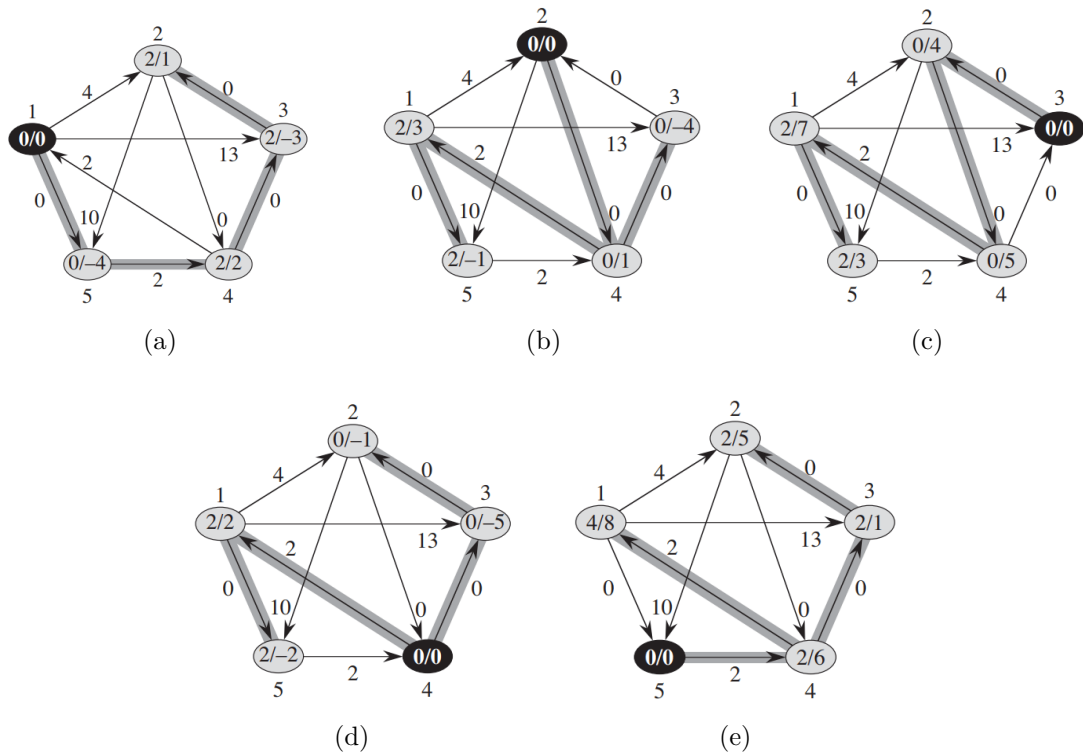


Figura 6: Rezultatul rulării lui *Dijkstra* pentru fiecare vârf din  $G$  folosind funcția pondere  $\hat{w}$ .

#### 4.6 Referințe

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. Introduction to Algorithms, Third Edition (3rd ed.). The MIT Press.
2. Geir Agnarsson and Raymond Greenlaw. 2006. Graph Theory: Modeling, Applications, and Algorithms. Prentice-Hall, Inc., Upper Saddle River, NJ, USA.
3. Mark Newman. 2010. Networks: An Introduction. Oxford University Press, Inc., New York, NY, USA.
4. Cristian A. Giumale. 2004. Introducere în analiza algoritmilor, teorie și aplicație. Polirom.